Semester Project - Database Systems Technology I

Database for a Rhythm Game

Andreas Brudovský

11. května 2026

Obsah

1 System Description

This project focuses on the design of a database layer for a computer rhythm game. In this game, users press keys in rhythm with music based on visual note prompts, earning points and trying to achieve the highest possible accuracy. The main purpose of the database is to manage dynamic data created by player interaction with the system.

The database covers the following key areas:

- **Content management**: Storing songs and their specific playable variants (difficulties).
- **Player data**: Managing user accounts, user roles (Player/Admin), and player statistics.
- **Game history**: Detailed records of each played session (score, accuracy, combo).
- **Gamification**: An achievement system motivating players to improve their performance.
- **Social interaction**: A comment system allowing feedback on songs.
- **Security and audit**: Tracking changes in sensitive user data through journaling.

1.1 Real-World Function of Individual Tables

In a real rhythm game implementation, the database would not be only a list of tables. It would represent a persistence layer for the whole game lifecycle: user registration, song selection, played sessions, results, comments, achievements, and audit history. The individual tables have the following practical roles:

- `tds_users` represents the basic user account. In practice, it would be used for login, e-mail management, password hash storage, and distinguishing whether a user is a player or an administrator.
- `tds_players` extends the user account with a player profile. It stores the public display name, total points, and level, which can be shown in player profiles, leaderboards, and statistics.
- `tds_admins` extends the user account with administrator permissions. In a real application, it would determine who can manage songs, handle reports, delete inappropriate comments, or moderate content.
- `tds_songs` is the song catalog. It stores the title, artist, BPM, duration, and optionally a cover image, which are needed on the song selection screen.
- `tds_song_variants` represents a specific playable version of a song, such as Easy, Hard, or Extreme. One song can have multiple variants, and each variant has its own note count and maximum score.
- `tds_game_sessions` stores one concrete played game session. In practice, a new row is created whenever a player finishes a song; it stores score, accuracy, maximum combo, and play time.
- `tds_achievements` defines the list of available achievements. These are reward definitions such as first played song, high accuracy, or large combo.
- `tds_user_achievements` connects players with achievements they have already unlocked. It serves as an unlocked achievement log and as an M:N relationship without additional relationship attributes.

- tds_comments stores player comments on songs. Through parent_comment_id, it also supports replies, so it can form a discussion tree.

- tds_content_reports stores moderation reports. A report can refer either to a song or to a comment, but never to both at the same time; this table demonstrates an ARC relationship.

- tds_historical_data stores audit history of changes. In a real system, it helps trace when a user's e-mail, username, or role changed and serves as a simple journaling table.

A typical application scenario works as follows: a player logs in through tds_users, the application loads the player's profile from tds_players, displays the catalog from tds_songs, and shows available difficulties from tds_song_variants. After finishing a song, the result is stored in tds_game_sessions; the system checks achievement conditions and, if necessary, inserts a record into tds_user_achievements. The player can write a comment into tds_comments; if the content is problematic, an administrator can handle it through tds_content_reports. If sensitive user data changes, a trigger stores the previous and new value in tds_historical_data.

1.2 DD S01 L02

Vysvětlete rozdíl mezi pojmem data a informace - příklad ve vašem projektu psáno anglicky

Data refers to raw, unorganized facts and figures that lack specific context. Information is data that has been processed, interpreted, and organized to provide meaning and value.

In my rhythm game database, an example of **data** is a single numerical *score* value (e.g., 85400) or an *accuracy* percentage (e.g., 95.5%) recorded in the `tds_game_sessions` table. An example of **information** is a calculated leaderboard showing the top 10 players, or a player's average accuracy across all songs, which provides context about their overall skill level.

1.3 DD S02 L02

Entity, instance, atributy a identifikátory - popište v příkladech na vašem projektu

Examples of entities, their attributes (including identifiers) and their instances in this database:

Users (tds_users) attributes:

- user_id (Identifier / Primary Key)
- username (VARCHAR2)
- email (VARCHAR2)
- role (CHAR)

Songs (tds_songs) attributes:

- song_id (Identifier / Primary Key)
- title (VARCHAR2)
- artist (VARCHAR2)
- bpm (NUMBER)

Instance example: We can have an instance of a user with an identifier `user_id` of 1, named "RhythmGod", with an email "rhythm@god.com"and a role of 'P' (Player). This user played an instance of a song with a `song_id` of 7, titled "Starlight", created by the artist "Muse", with a BPM of 122.

1.4 DD S03 L01

Popište všechny vztahy ve vaší databázi v angličtině, včetně kardinality a povinnosti členství – každý vztah dvěma větami (stránka 10)

- **Players and Game Sessions (1:N):** A single player can play multiple game sessions. Each game session must be played by exactly one player (mandatory for the session, optional for the player).
- **Songs and Variants (1:N):** One song can have multiple difficulty variants associated with it. Each variant must belong to exactly one song (mandatory for the variant, optional for the song).
- **Song Variants and Game Sessions (1:N):** A specific song variant can be played in many game sessions. Every game session must be tied to exactly one song variant (mandatory for the session,

optional for the variant).

- **Players and User Achievements (1:N):** A player can earn many achievements over time. Each earned achievement record must belong to exactly one player (mandatory for the record, optional for the player).

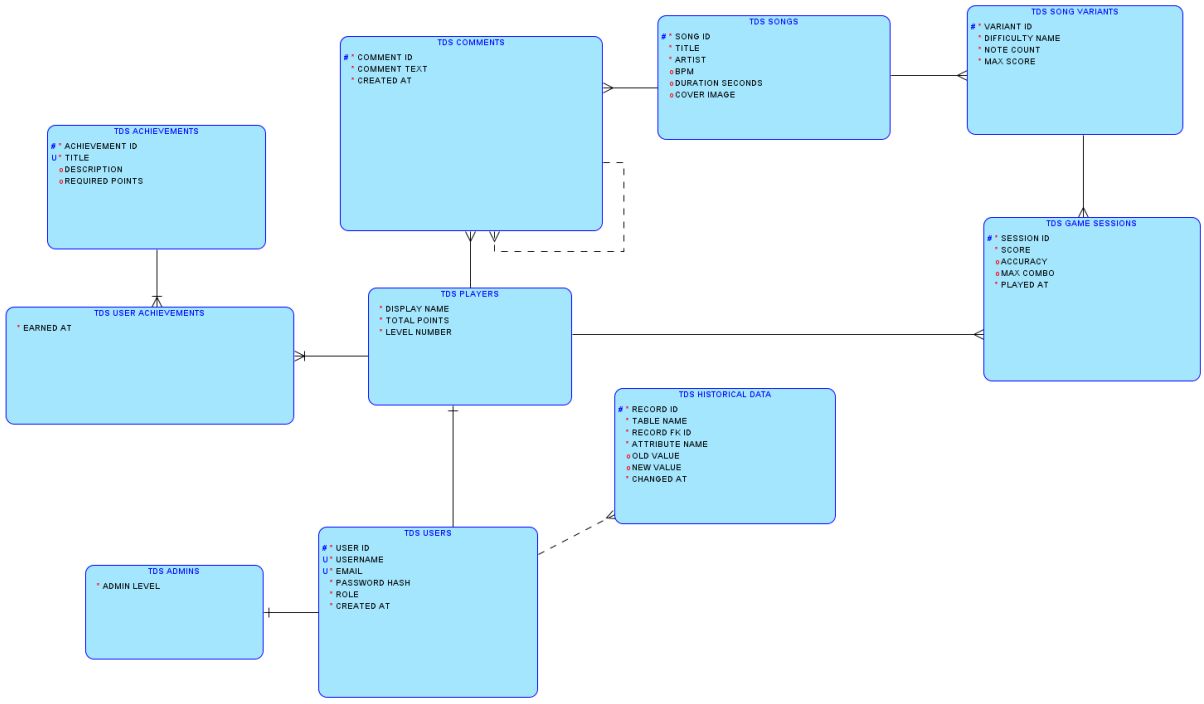
- **Achievements and User Achievements (1:N):** A specific achievement can be earned by many different players. Each earned achievement record must refer to exactly one achievement (mandatory for the record, optional for the achievement).

- **Comments and Replies (1:N Recursive):** A single comment can have multiple reply comments nested under it. Each reply comment can have a maximum of one parent comment (optional on both sides).

- **Users and Historical Data (1:N):** A user may have multiple records of their data changes logged over time. Each historical record conceptually relates to exactly one user (optional for the user).

1.5 DD S03 L02

Nakreslete ER diagram dle konvencí



Obrázek 1: Conceptual data model (ERD)

1.6 DD S30 L04

Maticový diagram se vztahy, nakreslete pro vaše řešení (stránka 5)

Entity	Players	Songs	Variants	Sessions	Achievements	Comments
Players	-	-	-	Plays	Earns	Writes
Songs	-	-	Has	-	-	Has
Variants	-	Belongs to	-	Played in	-	-
Sessions	Played by	-	Uses	-	-	-
Achievements	Earned by	-	-	-	-	-
Comments	Written by	Belongs to	-	-	-	Replies to (Recursive)

Tabulka 1: Matrix diagram of key relationships

1.7 DD S04 L01

Supertypy a podtypy - definujte minimálně jeden případ supertypu a subtypu ve vašem projektu

In this database, the `tds_users` table acts as a **supertype**. It contains shared attributes applicable to everyone (username, email, password_hash, role). There are two **subtypes** derived from this:

- `tds_players`: Attributes specific to regular gamers (total_points, level_number).

- `tds_admins`: Attributes specific to administrators (admin_level).

1.8 DD S04 L02

Popis obchodních pravidel k vašemu projektu

Data in the database must adhere to the following business rules:

- A user's role must be strictly either 'P' (Player) or 'A' (Admin).

- A song's BPM must be greater than 0.

- The accuracy of a game session must be between 0 and 100 percent.

- Total points earned by a player cannot be negative.

1.9 DD S05 L01

Zakomponujte do vašeho projektu alespoň jednu přenositelnou a jednu nepřenositelnou vazbu

Transferable relationship: The link between `tds_players` and `tds_comments`. Authorship of a comment could theoretically be transferred between user IDs.

Non-transferable relationship: The link between `tds_songs` and `tds_song_variants`. A specific difficulty chart is fundamentally tied to its parent song and cannot be moved to another.

1.10 DD S05 L03

Mějte ve svém projektu minimálně jednu vazbu M:N bez informace a jednu vazbu M:N s informací

M:N without information: `tds_user_achievements` links players and achievements without extra attributes.

M:N with information: `tds_game_sessions` links players and variants while storing *score*, *accuracy*, and *max_combo*.

1.11 DD S06 L01

Zakomponujte do svého projektu minimálně jednu identifikující vazbu 1:N s tím, že přenesený cizí klíč bude i klíčem v tabulce nové

An identifying 1:N relationship exists between `tds_players` and `tds_user_achievements`. One player can have many achievement records, and each achievement record must belong to exactly one player. The transferred foreign key `user_id` references `tds_players(user_id)` and is also part of the primary key of the new table `tds_user_achievements`.

A second identifying 1:N relationship exists between `tds_achievements` and `tds_user_achievements`. One achievement can be assigned to many players, and each assignment must refer to exactly one achievement. The transferred foreign key `achievement_id` references `tds_achievements(achievement_id)` and is also part of the primary key of `tds_user_achievements`.

1.12 DD S06 L02-04

Mějte své schéma v první normální formě – bez neatomických atributů. Ve druhé normální formě – bez vazeb na podklíči. Ve třetí normální formě – bez vazeb mezi sekundárními atributy

The schema complies with **1NF**, because all attributes contain atomic values only. For example, one game result is stored as one row in `tds_game_sessions` and values such as `score`, `accuracy` and `max_combo` are stored in separate columns, not as a list in one attribute.

The schema complies with **2NF**, because non-key attributes depend on the whole primary key. In the junction table `tds_user_achievements`, the primary key is composed of `user_id` and `achievement_id`. The table is intentionally kept without descriptive attributes such as player name or achievement title, because those depend only on one part of the key and are stored in `tds_players`/`tds_users` and `tds_achievements`.

The schema complies with **3NF**, because non-key attributes do not depend on other non-key attributes. For example, song information is stored in `tds_songs`, variant information in `tds_song_variants`, and session performance data in `tds_game_sessions`. This avoids transitive dependencies such as storing song title or artist directly in every game session.

1.13 DD S07 L01

Vyzkoušejte ve vašem projektu definovat ARC (lze definovat v ORACLE SQL Developeru Data Modeleru)

An ARC is an exclusive relationship rule where one record can be related to exactly one of several possible parent entities. In this project, an ARC can be defined for moderation reports: a report can refer either to a song or to a comment, but never to both at the same time.

```
CREATE TABLE tds_content_reports (
    report_id  NUMBER GENERATED BY DEFAULT AS IDENTITY,
    song_id    NUMBER,
    comment_id NUMBER,
    reason     VARCHAR2(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT pk_tds_content_reports PRIMARY KEY (report_id),
```

```
    CONSTRAINT fk_reports_song FOREIGN KEY (song_id)
        REFERENCES tds_songs(song_id),
    CONSTRAINT fk_reports_comment FOREIGN KEY (comment_id)
        REFERENCES tds_comments(comment_id),
    CONSTRAINT chk_reports_arc CHECK (
        (song_id IS NOT NULL AND comment_id IS NULL) OR
        (song_id IS NULL AND comment_id IS NOT NULL)
    )
);
```

The constraint `chk_reports_arc` implements the ARC behavior: exactly one of the two foreign keys must be filled.

1.14 DD S07 L02

Zkuste ve vašem projektu definovat hierarchické a rekurzivní relace

A recursive relationship is implemented in `tds_comments` via `parent_comment_id`. A comment can be a root comment, or it can be a reply to another comment in the same table. This creates a comment hierarchy.

```
SELECT LEVEL,
       LPAD('␣', 2 * (LEVEL - 1)) || comment_text AS comment_tree
FROM tds_comments
START WITH parent_comment_id IS NULL
CONNECT BY PRIOR comment_id = parent_comment_id;
```

A hierarchical relationship also exists in the content structure: one song has many variants and one variant can have many game sessions. The recursive comment structure is the strongest example because it can be queried directly with `START WITH`, `CONNECT BY PRIOR` and `LEVEL`.

1.15 DD S07 L03

Popište, jak ve vašem systému evidujete historická data

Historical data is managed in `tds_historical_data`. A trigger logs changes to sensitive user information.

1.16 DD S09 L01

Demonstrujte na vašem projektu ukládání změn v čase

Changes over time are demonstrated by storing the old and new values of changed user attributes in `tds_historical_data`. Each record contains the changed table name, record identifier, attribute name, previous value, new value and timestamp of the change.

1.17 DD S09 L02

Vyzkoušejte si žurnálování ve vašem projektu, tedy ukládání dřívějších historických dat (například změny platů, změny pracovišť atd.)

Journaling is implemented by a trigger on `tds_users`. When selected user data changes, the trigger inserts the previous and current value into `tds_historical_data`.

```
CREATE OR REPLACE TRIGGER trg_users_history
BEFORE UPDATE ON tds_users
FOR EACH ROW
BEGIN
    IF :OLD.email != :NEW.email THEN
        INSERT INTO tds_historical_data (table_name, record_fk_id,
            attribute_name, old_value, new_value)
        VALUES ('TDS_USERS', :OLD.user_id, 'EMAIL', :OLD.email, :NEW.
            email);
    END IF;
END;
/
```

1.18 DD S10 L01

Revidujte váš návrh podle konvencí pro čitelnost vašeho schématu

The schema follows naming conventions (prefix `tds_`), uses consistent primary key naming, and the diagram is organized to minimize line crossing for better readability.

1.19 DD S10 L02

Generické modelování – zvažte, případně popište nebo použijte ve svém řešení generický model datových struktur, v čem je tento přístup výhodnější oproti tradičním metodám návrhu datových struktur

The `tds_historical_data` table uses generic columns (`table_name`, `record_fk_id`) allowing it to store history for any entity, making the system highly scalable.

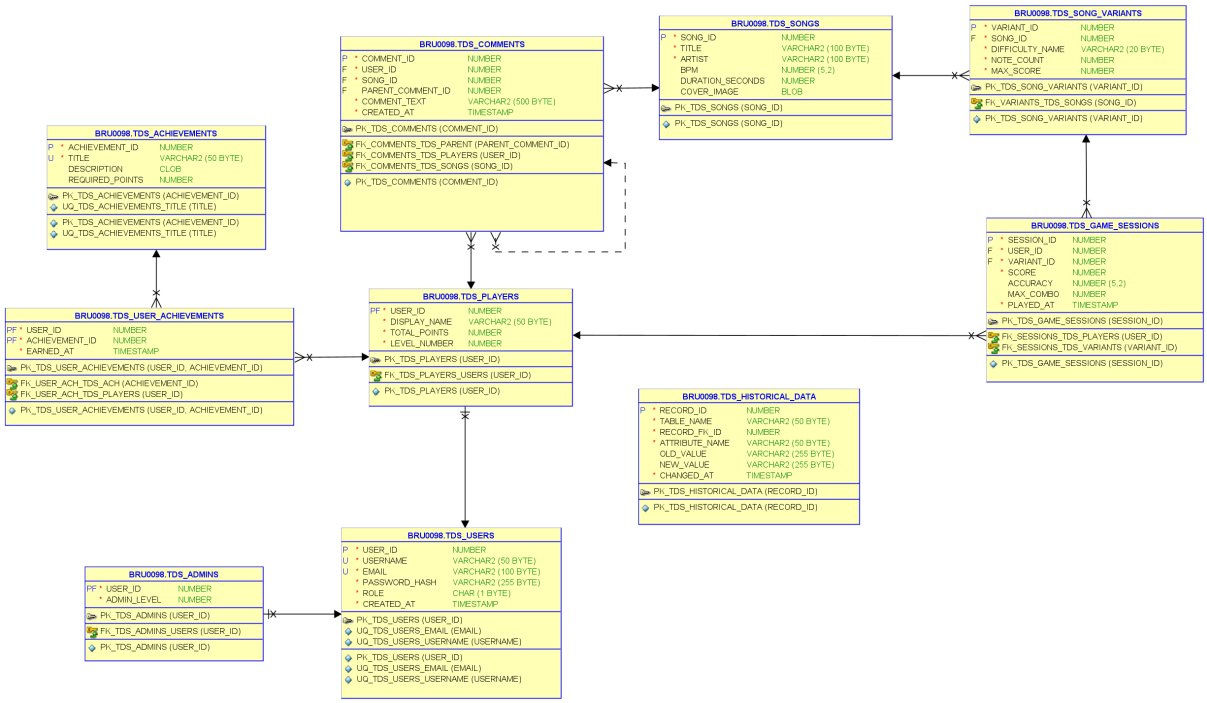
1.20 DD S11 L01

Popište na vašem projektu příklady integritních omezení pro entity, vazby, atributy a uživatelsky definované integrity

The system uses Primary Keys (Entity integrity), Foreign Keys (Referential integrity), and CHECK constraints (User-defined integrity, e.g., `accuracy <= 100`).

1.21 DD S11 L02-04

Generujte relační schéma z vašeho konceptuálního modelu a uvědomte si změny, které ve schématu nastaly a proč



Obrázek 2: Relational data model

1.22 DDL Script and Test Data (DML)

```
-- Table creation from the previous DDL part is intentionally omitted
   here
-- to save space in the document. The deployed SQL files contain the
   full DDL.
-- This block shows test data insertion:

INSERT INTO tds_users (username, email, password_hash, role) VALUES ('
   RhythmGod', 'god@rhythm.cz', 'hash1', 'P');
INSERT INTO tds_users (username, email, password_hash, role) VALUES ('
   NoobMaster', 'noob@rhythm.cz', 'hash2', 'P');
INSERT INTO tds_users (username, email, password_hash, role) VALUES ('
   AdminJoe', 'admin@rhythm.cz', 'hash3', 'A');

INSERT INTO tds_players (user_id, display_name, total_points,
   level_number) VALUES (1, 'God Mode', 50000, 42);
INSERT INTO tds_players (user_id, display_name, total_points,
   level_number) VALUES (2, 'Noobie', 1500, 5);

INSERT INTO tds_admins (user_id, admin_level) VALUES (3, 5);

INSERT INTO tds_songs (title, artist, bpm, duration_seconds) VALUES ('
   Starlight', 'Muse', 122, 240);
INSERT INTO tds_songs (title, artist, bpm, duration_seconds) VALUES ('
   Through the Fire', 'DragonForce', 200, 442);

INSERT INTO tds_song_variants (song_id, difficulty_name, note_count,
   max_score) VALUES (1, 'Easy', 200, 20000);
INSERT INTO tds_song_variants (song_id, difficulty_name, note_count,
   max_score) VALUES (1, 'Hard', 500, 50000);
INSERT INTO tds_song_variants (song_id, difficulty_name, note_count,
   max_score) VALUES (2, 'Extreme', 2500, 250000);

INSERT INTO tds_game_sessions (user_id, variant_id, score, accuracy,
   max_combo) VALUES (1, 3, 245000, 98.5, 2400);
INSERT INTO tds_game_sessions (user_id, variant_id, score, accuracy,
   max_combo) VALUES (2, 1, 15000, 85.0, 150);

INSERT INTO tds_achievements (title, description, required_points)
   VALUES ('First Steps', 'Play your first song.', 0);
INSERT INTO tds_achievements (title, description, required_points)
   VALUES ('Godlike', 'Get over 98% accuracy on Extreme.', 10000);

INSERT INTO tds_user_achievements (user_id, achievement_id) VALUES (1,
   1);
INSERT INTO tds_user_achievements (user_id, achievement_id) VALUES (1,
   2);

INSERT INTO tds_comments (user_id, song_id, parent_comment_id,
   comment_text) VALUES (1, 2, NULL, 'This song is too fast!');
INSERT INTO tds_comments (user_id, song_id, parent_comment_id,
   comment_text) VALUES (2, 2, 1, 'I agree, my fingers hurt.');
COMMIT;
```

1.23 DD S15 L01

Napište dotaz pro spojování řetězců pomocí ||, pomocí CONCAT(). SELECT DISTINCT

```sql
SELECT DISTINCT u.username || ' - ' || p.display_name AS full_identifier
    ,
        CONCAT('Email: ', u.email) AS contact_info
FROM tds_users u JOIN tds_players p ON u.user_id = p.user_id;
```

1.24 DD S16 L02

WHERE podmínky pro výběr řídků. Funkce LOWER, UPPER, INITCAP

```sql
SELECT UPPER(title) AS song_title, INITCAP(artist) AS artist_name, LOWER
    (difficulty_name) AS diff
FROM tds_songs s JOIN tds_song_variants v ON s.song_id = v.song_id
WHERE bpm > 150;
```

1.25 DD S16 L03

BETWEEN ... AND, LIKE (%,_), IN(), IS NULL, IS NOT NULL

```sql
SELECT * FROM tds_game_sessions
WHERE accuracy BETWEEN 90.0 AND 100.0
  AND max_combo IS NOT NULL
  AND user_id IN (1, 2, 3);

SELECT * FROM tds_users WHERE email LIKE '%@rhythm.cz';
```

1.26 DD S17 L01

AND, OR, NOT. Priorita vyhodnocení pomocí ()

```sql
SELECT * FROM tds_song_variants
WHERE (note_count > 1000 AND max_score > 100000) OR NOT difficulty_name
    = 'Easy';
```

1.27 DD S17 L02

ORDER BY atr [ASC/DESC]. Třídění podle jednoho nebo více atributů

```sql
SELECT * FROM tds_game_sessions
ORDER BY score DESC, accuracy DESC, played_at ASC;
```

1.28 DD S17 L03

Jednořádkové funkce. Víceřádkové funkce MIN, MAX, AVG, SUM, COUNT

```sql
SELECT user_id, COUNT(session_id) as games_played, MAX(score) as
    best_score,
        MIN(accuracy) as worst_acc, AVG(accuracy) as avg_acc, SUM(
            max_combo) as total_combo
FROM tds_game_sessions
GROUP BY user_id;
```

1.29 SQL S01 L01

LOWER, UPPER, INITCAP. CONCAT, SUBSTR, LENGTH, INSTR, LPAD, RPAD, TRIM, REPLACE. Použijte tabulku DUAL

```
SELECT SUBSTR(username, 1, 5) AS short_name, LENGTH(email) AS email_len,
       INSTR(email, '@') AS at_position, LPAD(role, 5, '*') AS
          padded_role,
       REPLACE(email, 'rhythm.cz', 'game.com') AS new_email
FROM tds_users;

SELECT TRIM('  Rhythm Game  ') AS trimmed FROM DUAL;
```

1.30 SQL S01 L02

ROUND, TRUNC zaukrouhlení na 2 desetinná místa, na celé tisíce MOD

```
SELECT score, ROUND(accuracy, 1) AS acc_rounded, TRUNC(accuracy) AS
   acc_truncated,
       MOD(max_combo, 10) AS combo_mod_10
FROM tds_game_sessions;
```

1.31 SQL S01 L03

MONTHS_BETWEEN, ADD_MONTHS, NEXT_DAY, LAST_DAY, ROUND, TRUNC. Systémová konstanta SYSDATE

```
SELECT created_at, ADD_MONTHS(created_at, 6) AS mid_anniversary,
       LAST_DAY(created_at) AS end_of_month,
       NEXT_DAY(SYSDATE, 'MONDAY') AS next_monday,
       ROUND(SYSDATE, 'MONTH') AS rounded_month,
       TRUNC(SYSDATE, 'YEAR') AS year_start,
       MONTHS_BETWEEN(SYSDATE, created_at) AS months_active
FROM tds_users;
```

1.32 SQL S02 L01

TO_CHAR, TO_NUMBER, TO_DATE

```
SELECT TO_CHAR(played_at, 'DD.MM.YYYY HH24:MI') AS formatted_date,
       TO_CHAR(score, '999,999,999') AS formatted_score,
       TO_NUMBER('12345') AS parsed_number
FROM tds_game_sessions;

SELECT TO_DATE('01.01.2024', 'DD.MM.YYYY') AS parsed_date FROM DUAL;
```

1.33 SQL S02 L02

NVL, NVL2, NULLIF, COALESCE

```
SELECT comment_id, NVL(parent_comment_id, 0) AS safe_parent_id,
       NVL2(parent_comment_id, 'Je to odpoved', 'Hlavni komentar') AS
          comment_type,
       COALESCE(parent_comment_id, comment_id) AS thread_id
FROM tds_comments;
```

1.34 SQL S02 L03

DECODE, CASE, IF-THEN-ELSE

```
SELECT username,
       CASE role
           WHEN 'A' THEN 'Administrator'
           WHEN 'P' THEN 'Player'
           ELSE 'Unknown'
       END AS role_desc,
       DECODE(role, 'A', 1, 0) AS is_admin_flag
FROM tds_users;
```

1.35 SQL S03 L01

NATURAL JOIN, CROSS JOIN

```
-- CROSS JOIN: every player combined with every song, useful for
   generated statistics
SELECT p.display_name, s.title
FROM tds_players p CROSS JOIN tds_songs s;

-- NATURAL JOIN: joins by common columns, for example user_id
SELECT * FROM tds_users NATURAL JOIN tds_players;
```

1.36 SQL S03 L02

JOIN ... USING(atr), JOIN.. ON (podmínka spojení)

```
SELECT u.username, p.display_name
FROM tds_users u JOIN tds_players p USING (user_id);

SELECT gs.score, sv.difficulty_name
FROM tds_game_sessions gs JOIN tds_song_variants sv ON gs.variant_id =
   sv.variant_id;
```

1.37 SQL S03 L03

LEFT OUTER JOIN ... ON (), RIGHT OUTER JOIN ... ON (), FULL OUTER JOIN ... ON ()

```
-- LEFT JOIN: all songs and their variants, including songs without
   variants
SELECT s.title, v.difficulty_name
FROM tds_songs s LEFT OUTER JOIN tds_song_variants v ON s.song_id = v.
   song_id;

-- RIGHT JOIN: all achievements and optionally players who have them
SELECT a.title, ua.user_id
FROM tds_user_achievements ua RIGHT OUTER JOIN tds_achievements a ON ua.
   achievement_id = a.achievement_id;

-- FULL JOIN: combination of both sides
SELECT u.username, p.display_name
FROM tds_users u FULL OUTER JOIN tds_players p ON u.user_id = p.user_id;
```

1.38 SQL S03 L04

Spojování 2x stejné tabulky s přejmenováním (vazba mezi nadřízenými a podřízenými v jedné tabulce). Hierarchické dotazování – stromová struktura zanoření START WITH, CONNECT BY PRIOR, LEVEL

```
-- Self-join
SELECT c1.comment_text AS Reply, c2.comment_text AS Original
FROM tds_comments c1 JOIN tds_comments c2 ON c1.parent_comment_id = c2.
    comment_id;

-- Hierarchical query: comment tree
SELECT LEVEL, LPAD('␣', 2*(LEVEL-1)) || comment_text AS tree_comment
FROM tds_comments
START WITH parent_comment_id IS NULL
CONNECT BY PRIOR comment_id = parent_comment_id;
```

1.39 SQL S04 L02

AVG, COUNT, MIN, MAX, SUM, VARIANCE, STDDEV

```
SELECT variant_id, AVG(accuracy) AS avg_acc, SUM(score) AS total_score,
       VARIANCE(accuracy) AS acc_var, STDDEV(accuracy) AS acc_stddev
FROM tds_game_sessions
GROUP BY variant_id;
```

1.40 SQL S04 L03

COUNT, COUNT(DISTINCT), NVL. Rozdíl mezi COUNT (*) a COUNT (atribut). Proč NVL u agregačních funkcí

```
-- COUNT(*) counts all rows, COUNT(parent_comment_id) counts only non-
    NULL parent IDs
SELECT COUNT(*) AS total_comments,
       COUNT(parent_comment_id) AS total_replies,
       COUNT(DISTINCT user_id) AS unique_commenters,
       AVG(NVL(parent_comment_id, 0)) AS avg_parent_id_safe
FROM tds_comments;
```

1.41 SQL S05 L01

GROUP BY, HAVING

```
SELECT variant_id, COUNT(*) as plays, AVG(score) as avg_score
FROM tds_game_sessions
GROUP BY variant_id
HAVING COUNT(*) > 0 AND AVG(score) > 10000;
```

1.42 SQL S05 L02

ROLLUP, CUBE, GROUPING SETS

```
-- Subtotals by user and variant, including the overall total
SELECT user_id, variant_id, SUM(score)
FROM tds_game_sessions
```

```sql
GROUP BY ROLLUP(user_id, variant_id);

SELECT user_id, variant_id, SUM(score)
FROM tds_game_sessions
GROUP BY CUBE(user_id, variant_id);

SELECT user_id, variant_id, SUM(score)
FROM tds_game_sessions
GROUP BY GROUPING SETS ((user_id), (variant_id), ());
```

1.43 SQL S05 L03

Množinové operace v SQL – UNION, UNION ALL, INTERSECT, MINUS. ORDER BY u množinových operací

```sql
-- Returns unique IDs of users who played a game or wrote comments
SELECT user_id FROM tds_game_sessions
UNION
SELECT user_id FROM tds_comments
ORDER BY user_id DESC;

SELECT user_id FROM tds_game_sessions
UNION ALL
SELECT user_id FROM tds_comments;

SELECT user_id FROM tds_game_sessions
INTERSECT
SELECT user_id FROM tds_comments;

-- Returns users who played a game but never wrote a comment
SELECT user_id FROM tds_game_sessions
MINUS
SELECT user_id FROM tds_comments;
```

1.44 SQL S06 L01

Vnořené dotazy. Výsledek jako jediná hodnota. Vícesloupcový poddotaz. EXISTS, NOT EXISTS

```sql
-- EXISTS example
SELECT display_name FROM tds_players p
WHERE EXISTS (SELECT 1 FROM tds_game_sessions s WHERE s.user_id = p.
   user_id);

-- Multi-column subquery: find sessions with the same score and combo as
    session 1
SELECT * FROM tds_game_sessions
WHERE (score, max_combo) = (SELECT score, max_combo FROM
   tds_game_sessions WHERE session_id = 1);
```

1.45 SQL S06 L02

Jednořádkové poddotazy

```sql
-- Player/session with the highest score overall
SELECT user_id, score FROM tds_game_sessions
WHERE score = (SELECT MAX(score) FROM tds_game_sessions);
```

1.46 SQL S06 L03

Víceřádkové poddotazy IN, ANY, ALL. NULL hodnoty v poddotazech

```
-- Players who played a variant harder than Easy
SELECT display_name FROM tds_players
WHERE user_id IN (
    SELECT user_id FROM tds_game_sessions
    WHERE variant_id IN (SELECT variant_id FROM tds_song_variants WHERE
       difficulty_name != 'Easy')
);

SELECT score FROM tds_game_sessions WHERE score > ALL (SELECT
   required_points FROM tds_achievements);

SELECT score FROM tds_game_sessions WHERE score > ANY (SELECT
   required_points FROM tds_achievements);

SELECT comment_id, comment_text
FROM tds_comments
WHERE parent_comment_id NOT IN (
    SELECT parent_comment_id
    FROM tds_comments
    WHERE parent_comment_id IS NOT NULL
);
```

1.47 SQL S06 L04

WITH.. AS() konstrukce poddotazu

```
WITH PlayerStats AS (
    SELECT user_id, AVG(accuracy) as avg_acc, SUM(score) as total_score
    FROM tds_game_sessions
    GROUP BY user_id
)
SELECT p.display_name, ps.avg_acc, ps.total_score
FROM tds_players p JOIN PlayerStats ps ON p.user_id = ps.user_id
WHERE ps.avg_acc > 90;
```

1.48 SQL S07 L01

INSERT INTO Tab VALUES(). INSERT INTo Tab (atr, atr) VALUES(). INSERT INTO Tab AS SELECT ...

```
INSERT INTO tds_achievements (title, description, required_points)
   VALUES ('Newbie', 'Register', 0);

-- Insert data from a SELECT statement, for example for archiving
CREATE TABLE tds_archived_sessions AS SELECT * FROM tds_game_sessions
   WHERE 1=0;
INSERT INTO tds_archived_sessions SELECT * FROM tds_game_sessions WHERE
   played_at < SYSDATE - 365;
```

1.49 SQL S07 L02

UPDATE Tab SET atr = ... WHERE podm. DELETE FROM Tab WHERE atr = ...

```
UPDATE tds_players SET total_points = total_points + 100 WHERE
    level_number < 10;

DELETE FROM tds_comments WHERE comment_text LIKE '%delete-me%';
```

1.50 SQL S07 L03

DEFAULT, MERGE, Multi-Table Inserts

```
MERGE INTO tds_players p
USING (SELECT user_id, SUM(score) as s FROM tds_game_sessions GROUP BY
    user_id) s
ON (p.user_id = s.user_id)
WHEN MATCHED THEN UPDATE SET p.total_points = p.total_points + s.s;

INSERT ALL
  INTO tds_users (username, email, password_hash) VALUES ('multitest', '
     mt@r.cz', 'pwd')
  INTO tds_players (user_id, display_name) VALUES (999, 'Multi') -- In
     practice this would use a trigger/sequence
SELECT * FROM DUAL;
```

1.51 SQL S08 L01

Objekty v databázi – Tabulky, Indexy, Constraint, View, Sequnce, Synonym. CREATE, ALTER, DROP, RENAME, TRUNCATE. CREATE TABLE (atr DAT TYP, DEFAULT NOT NULL). ORGANIZATION EXTERNAL, TYPE ORACLE_LOADER, DEFAULT DICTIONARY, ACESS PARAMETERS, RECORDS DELIMITED BY NEWLINE, FIELDS, LOCATION

```
CREATE TABLE tds_temp_logs (log_id NUMBER, text VARCHAR2(100));
ALTER TABLE tds_temp_logs ADD (log_date DATE);
RENAME tds_temp_logs TO tds_system_logs;
TRUNCATE TABLE tds_system_logs;
DROP TABLE tds_system_logs;

-- External table syntax: requires a DIRECTORY object in the database
/*
CREATE TABLE ext_songs (title VARCHAR2(100), bpm NUMBER)
ORGANIZATION EXTERNAL (
  TYPE ORACLE_LOADER DEFAULT DIRECTORY ext_dir
  ACCESS PARAMETERS (RECORDS DELIMITED BY NEWLINE FIELDS TERMINATED BY
     ',')
  LOCATION ('songs.csv')
);
*/
```

1.52 SQL S08 L02

TIMESTAMP, TIMESTAMP WITH TIME ZONE, TIMESTAM WITH LOCAL TIMEZONE. INTERVAL YEAT TO MONTH, INTERVAL DAY TO SECOND. CHAR, VARCHAR2, CLOB. NUMBER. BLOB

```
CREATE TABLE tds_datatypes_demo (
    id NUMBER,
    event_time TIMESTAMP WITH TIME ZONE,
    duration INTERVAL DAY TO SECOND,
```

```
    long_desc CLOB,
    binary_file BLOB
);
```

1.53 SQL S08 L03

ALTER TABLE (ADD, MODIFY, DROP), DROP, RENAME. FLASHBACK TABLE Tab TO BEFORE DROP (pohled USER_RECYCLEBIN). DELETE, TRUNCATE. COMMENT ON TABLE. SET UNUSED

```
COMMENT ON TABLE tds_game_sessions IS 'Stores data about each played
   song by a user';
ALTER TABLE tds_users SET UNUSED (password_hash); -- Hides the column

-- Flashback example, if recycle bin is enabled
-- DROP TABLE tds_datatypes_demo;
-- FLASHBACK TABLE tds_datatypes_demo TO BEFORE DROP;
```

1.54 SQL S10 L01

CREATE TABLE (NOT NULL A UNIQUE constraint). CREATE TABLE Tab AS SELECT ... Vlastní vs. systémové pojmenování CONSTRAINT podmínek

```
-- Custom constraint naming
CREATE TABLE tds_test (
    id NUMBER CONSTRAINT pk_tds_test PRIMARY KEY,
    name VARCHAR2(50) CONSTRAINT uq_tds_test UNIQUE
);
```

1.55 SQL S10 L02

CONSTRAINT – NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY (atr REFERENCES Tab(atr)), CHECK. Cizí klíče, ON DELETE, ON UPDATE, RESTRICT, CASCADE atd.

```
-- These constraints are already applied in detail in the main DDL
   script
-- (napr. CONSTRAINT fk_sessions_tds_players FOREIGN KEY (user_id)
   REFERENCES tds_players(user_id) ON DELETE CASCADE)
```

1.56 SQL S10 L03

USER_CONSTRAINTS

```
SELECT constraint_name, constraint_type, table_name
FROM user_constraints
WHERE table_name = 'TDS_USERS';
```

1.57 SQL S11 L01

CREATE VIEW. FORCE, NOFORCE. WITCH CHECK OPTION. WITH READ ONLY. Simple vs. Compex VIEW

```sql
CREATE OR REPLACE NOFORCE VIEW v_player_summary AS
SELECT user_id, display_name, total_points, level_number
FROM tds_players;

CREATE OR REPLACE FORCE VIEW v_high_scores AS
SELECT session_id, user_id, score, accuracy
FROM tds_game_sessions
WHERE accuracy >= 90
WITH CHECK OPTION;

CREATE OR REPLACE VIEW v_top_players AS
SELECT p.display_name, p.total_points, p.level_number
FROM tds_players p
WHERE p.total_points > 1000
WITH READ ONLY;
```

1.58 SQL S11 L03

INLINE VIEW Poddotaz v podobě tabulky SELECT atr FROM (SELECT * FROM Tab) alt_tab

```sql
-- Inline View
SELECT top.display_name FROM (SELECT display_name, total_points FROM
    tds_players ORDER BY total_points DESC) top WHERE ROWNUM <= 5;
```

1.59 SQL S12 L01

CREATE SEQUENCE nazev INCREMENT BY n START WITH m, (NO)MAXVALUE, (NO)MINVALUE, (NO)CYCLE, (NO)CACHE. ALTER SEQUENCE

```sql
CREATE SEQUENCE seq_tds_custom START WITH 100 INCREMENT BY 5 NOCACHE
    NOCYCLE;

ALTER SEQUENCE seq_tds_custom INCREMENT BY 10 NOCACHE NOCYCLE;
```

1.60 SQL S12 L02

CREATE INDEX, PRIMARY KEY, UNIQUE KEY, FOREIGN KEY

```sql
CREATE INDEX idx_tds_game_sessions_acc ON tds_game_sessions(accuracy
    DESC);
```

1.61 SQL S13 L01

GRANT ... ON ... TO ... PUBLIC. REVOKE. Jaká práva lze přidělit na jaké objekty? (ALTER, DELETE, EXECUTE, INDEX, INSERT, REFERENCES, SELECT, UPDATE) – (TABLE, VIEW, SEQUENCE, PROCEDURE)

```sql
GRANT SELECT, INSERT, DELETE ON tds_game_sessions TO PUBLIC;
REVOKE DELETE ON tds_game_sessions FROM PUBLIC;
```

1.62 SQL S13 L03

Regulární výrazy. REGEXP_LIKE, REGEXP_REPLACE, REGEXP_INSTR, REGEXP_SUBSTR, REGEXP_COUNT

```
-- Finds e-mails containing the word rhythm
SELECT email FROM tds_users WHERE REGEXP_LIKE(email, '^.*rhythm.*$');

SELECT REGEXP_REPLACE(email, '@.*', '@hidden.com') AS masked_email FROM
   tds_users;

SELECT title,
       REGEXP_INSTR(title, '[[:alpha:]]+') AS first_word_position,
       REGEXP_SUBSTR(title, '[[:alpha:]]+') AS first_word,
       REGEXP_COUNT(title, '[[:alpha:]]+') AS word_count
FROM tds_songs;
```

1.63 SQL S14 L01

Transakce, COMMIT, ROLLBACK, SAVEPOINT

```
UPDATE tds_players SET total_points = 0;
SAVEPOINT before_delete;
DELETE FROM tds_players;
ROLLBACK TO before_delete;
COMMIT;
```

1.64 SQL S15 L01

Alternativní zápis spojování bez JOIN s podmínkou spojení ve WHERE. Levé a pravé spojení s pomocí atrA = atrB (+)

```
-- Old Oracle-style left outer join
SELECT s.title, gs.score
FROM tds_songs s, tds_song_variants v, tds_game_sessions gs
WHERE s.song_id = v.song_id(+)
  AND v.variant_id = gs.variant_id(+);

-- Old Oracle-style right outer join
SELECT u.username, p.display_name
FROM tds_users u, tds_players p
WHERE u.user_id(+) = p.user_id;
```

1.65 SQL S16 L03

Rekapitulace příkazů a parametrů – vše co nebylo uvedeno v předchozích bodech dodělat zde

```
-- Complex recap query:
-- joins users and their game sessions, calculates statistics for the
   last year,
-- and filters only users with activity.
SELECT
    u.username,
    COUNT(gs.session_id) AS total_plays,
    MAX(gs.score) AS highest_score,
```

```
    ROUND(AVG(gs.accuracy), 2) AS average_accuracy
FROM tds_users u
JOIN tds_game_sessions gs ON u.user_id = gs.user_id
WHERE gs.played_at >= ADD_MONTHS(SYSDATE, -12)
GROUP BY u.username
HAVING COUNT(gs.session_id) > 0
ORDER BY highest_score DESC;
```